

QUERIES

1: Retrieve All Traders

```
SELECT * FROM Trader;
```

2: Traders with Balance Above ₹1,00,000

```
SELECT Trader_ID, Name, Email, Balance  
FROM Trader  
WHERE Balance > 100000.00;
```

3: Orders with Specific Statuses (IN operator)

```
SELECT order_id, trader_id, stock_id, quantity, status,  
timestamp  
FROM orders  
WHERE status IN ('PENDING', 'CANCELLED');
```

4: Orders Placed Within a Date Range (BETWEEN)

```
SELECT Order_ID, Trader_ID, Order_Type, Quantity, Status  
FROM orders  
WHERE Timestamp BETWEEN '2024-11-01 00:00:00' AND  
'2024-11-02 23:59:59'  
ORDER BY Timestamp ASC;
```

5: Pattern Matching with LIKE

```
SELECT order_id, trader_id, order_type, quantity, status
FROM orders
WHERE timestamp >= CURRENT_DATE
ORDER BY timestamp;
```

6: Traders Ranked by Balance (Descending)

```
SELECT Trader_ID, Name, Balance
FROM Trader
ORDER BY Balance DESC;
```

7: Multi-Column Sort — Orders by Status then Timestamp

```
SELECT Order_ID, Status, Order_Type, Timestamp
FROM orders
ORDER BY Status ASC, Timestamp DESC;
```

8: Total Orders Placed per Trader

```
SELECT t.Trader_ID, t.Name, COUNT(o.Order_ID) AS
Total_Orders
FROM Trader t
JOIN orders o ON t.Trader_ID = o.Trader_ID
GROUP BY t.Trader_ID, t.Name
ORDER BY Total_Orders DESC;
```

9: Total Traded Value and Count per Stock

```
SELECT ss.Symbol, ss.Company_Name,  
SUM(t.Execution_Price * t.Quantity) AS Total_Value,  
COUNT(t.Trade_ID) AS Trade_Count  
FROM Trade t  
JOIN Trade_Order tord ON t.Trade_ID = tord.Trade_ID  
JOIN orders o ON tord.Order_ID = o.Order_ID  
JOIN Stock_Base sb ON o.Stock_ID = sb.Stock_ID  
JOIN Stock_Symbol ss ON sb.Symbol = ss.Symbol  
GROUP BY ss.Symbol, ss.Company_Name  
ORDER BY Total_Value DESC;  
  
ORDER BY o.Timestamp;
```

10: Price Statistics per Stock (AVG, MIN, MAX)

```
SELECT ss.Symbol, ROUND(AVG(t.Execution_Price), 2) AS  
Avg_Price, MIN(t.Execution_Price) AS Min_Price,  
MAX(t.Execution_Price) AS Max_Price  
FROM Trade t  
JOIN Trade_Order tord ON t.Trade_ID = tord.Trade_ID  
JOIN orders o ON tord.Order_ID = o.Order_ID  
JOIN Stock_Base sb ON o.Stock_ID = sb.Stock_ID  
JOIN Stock_Symbol ss ON sb.Symbol = ss.Symbol  
GROUP BY ss.Symbol;
```

11: HAVING — Traders with More Than 2 Orders

```
SELECT t.Trader_ID, t.Name,  
COUNT(o.Order_ID) AS Order_Count  
FROM Trader t
```

```
JOIN orders o ON t.Trader_ID = o.Trader_ID
GROUP BY t.Trader_ID, t.Name
HAVING COUNT(o.Order_ID) > 2
ORDER BY Order_Count DESC;
```

12: INNER JOIN — Full Order Book Detail

```
SELECT o.Order_ID,
       t.Name AS Trader_Name,
       ss.Symbol,
       ss.Company_Name,
       o.Quantity,
       o.Limit_Price,
       o.Order_Type,
       o.Status,
       o.Timestamp
FROM orders o
JOIN Trader t ON o.Trader_ID = t.Trader_ID
JOIN Stock_Base sb ON o.Stock_ID = sb.Stock_ID
JOIN Stock_Symbol ss ON sb.Symbol = ss.Symbol
ORDER BY o.Timestamp;
```

13: LEFT JOIN — All Traders Including Those with No Portfolio

```
SELECT t.Trader_ID, t.Name,
       COALESCE(ss.Symbol, 'No Holdings') AS Symbol,
       COALESCE(p.Quantity, 0) AS Shares_Held
```

```
FROM Trader t
LEFT JOIN Portfolio p ON t.Trader_ID = p.Trader_ID
LEFT JOIN Stock_Base sb ON p.Stock_ID = sb.Stock_ID
LEFT JOIN Stock_Symbol ss ON sb.Symbol = ss.Symbol
ORDER BY t.Trader_ID, ss.Symbol;
```

14: RIGHT JOIN — All Stocks Including Those Never Traded

```
SELECT ss.Symbol, ss.Company_Name,
       COUNT(o.Order_ID) AS Times_Ordered
FROM orders o
RIGHT JOIN Stock_Base sb ON o.Stock_ID = sb.Stock_ID
RIGHT JOIN Stock_Symbol ss ON sb.Symbol = ss.Symbol
GROUP BY ss.Symbol, ss.Company_Name
ORDER BY Times_Ordered DESC;
```

15: Buyer-Seller Pair Reconstruction via Trade_Order Bridge

```
SELECT tr.Trade_ID,
       t_buy.Name AS Buyer,
       t_sell.Name AS Seller,
       tr.Execution_Price,
       tr.Quantity,
       tr.Trade_Time
FROM Trade tr
JOIN Trade_Order to1 ON tr.Trade_ID = to1.Trade_ID
JOIN orders buy_o ON to1.Order_ID = buy_o.Order_ID
  AND buy_o.Order_Type = 'BUY'
JOIN Trader t_buy ON buy_o.Trader_ID = t_buy.Trader_ID
```

```
JOIN Trade_Order to2 ON tr.Trade_ID = to2.Trade_ID
JOIN orders sell_o ON to2.Order_ID = sell_o.Order_ID
  AND sell_o.Order_Type = 'SELL'
JOIN Trader t_sell ON sell_o.Trader_ID = t_sell.Trader_ID
ORDER BY tr.Trade_Time;
```

16: CROSS JOIN — All Possible Trader-Stock Watch Combinations

```
SELECT t.Name AS Trader_Name, ss.Symbol,
ss.Company_Name
FROM Trader t
CROSS JOIN Stock_Symbol ss
ORDER BY t.Name, ss.Symbol;
```

17: Scalar Subquery — Traders with Above-Average Balance

```
SELECT Trader_ID, Name, Balance
FROM Trader
WHERE Balance > (SELECT AVG(Balance) FROM Trader)
ORDER BY Balance DESC;
```

18: Subquery with IN — Traders Who Have at Least One FILLED Order

```
SELECT Trader_ID, Name, Email
FROM Trader
WHERE Trader_ID IN (
```

```
SELECT DISTINCT Trader_ID FROM orders
WHERE Status = 'FILLED'
);
```

19: Subquery with NOT IN — Traders Who Have Never Traded

```
SELECT Trader_ID, Name, Email, Balance
FROM Trader
WHERE Trader_ID NOT IN (
    SELECT DISTINCT Trader_ID FROM orders
    WHERE Status = 'FILLED'
);
```

20: Correlated Subquery — Each Trader's Highest Single Order Quantity

```
SELECT o.Order_ID, t.Name AS Trader_Name,
       o.Quantity, o.Order_Type, o.Status
FROM orders o
JOIN Trader t ON o.Trader_ID = t.Trader_ID
WHERE o.Quantity = (
    SELECT MAX(o2.Quantity)
    FROM orders o2
    WHERE o2.Trader_ID = o.Trader_ID
);
```

21: EXISTS Subquery — Stocks That Have Been Ordered At Least Once

```
SELECT ss.Symbol, ss.Company_Name
FROM Stock_Symbol ss
WHERE EXISTS (
    SELECT 1
    FROM Stock_Base sb
    JOIN orders o ON sb.Stock_ID = o.Stock_ID
    WHERE sb.Symbol = ss.Symbol
);
```

22: UNION — Combined Email Directory of Traders and Administrators

```
SELECT Email, 'Trader' AS User_Role FROM Trader
UNION
SELECT Email, 'Administrator' AS User_Role FROM
Administrator
ORDER BY Email;
```

23: UNION — Combined Email Directory of Traders and Administrators

```
SELECT DISTINCT t.Trader_ID, t.Name
FROM Trader t
WHERE t.Trader_ID IN (
    SELECT DISTINCT Actor_ID FROM System_Audit_Log
)
ORDER BY t.Trader_ID;
```


24: String Functions — Format Trader Display Names

```
SELECT Trader_ID,  
       UPPER(Name) AS Name_Upper,  
       LOWER(Email) AS Email_Lower,  
       CONCAT(Name, ' <', Email, '>') AS Display_String,  
       LENGTH(Name) AS Name_Length  
FROM Trader;
```

25: Date Functions — Decompose Audit Log Timestamps

```
SELECT log_id,  
       action_type,  
       EXTRACT(YEAR FROM action_timestamp) AS year,  
       EXTRACT(MONTH FROM action_timestamp) AS month,  
       EXTRACT(DAY FROM action_timestamp) AS day,  
       TO_CHAR(action_timestamp, 'Day') AS day_name,  
       TO_CHAR(action_timestamp, 'DD Month YYYY') AS  
formatted_date  
FROM system_audit_log  
ORDER BY action_timestamp;
```

26: DATEDIFF — Days Since Each Trader's Last Order

```
SELECT t.trader_id,  
       t.name,  
       MAX(o.timestamp) AS last_order_date,  
       CURRENT_DATE - MAX(o.timestamp)::date AS  
days_since_last_order
```

```
FROM trader t
JOIN orders o ON t.trader_id = o.trader_id
GROUP BY t.trader_id, t.name
ORDER BY days_since_last_order ASC;
```

27: CREATE VIEW — Trader Portfolio Summary

```
CREATE OR REPLACE VIEW vw_Trader_Portfolio AS
SELECT t.Trader_ID,
       t.Name AS Trader_Name,
       ss.Symbol,
       ss.Company_Name,
       p.Quantity AS Shares_Held
FROM Portfolio p
JOIN Trader t ON p.Trader_ID = t.Trader_ID
JOIN Stock_Base sb ON p.Stock_ID = sb.Stock_ID
JOIN Stock_Symbol ss ON sb.Symbol = ss.Symbol;
```

28: CREATE VIEW — Live Order Book (PENDING and OPEN orders only)

```
CREATE OR REPLACE VIEW vw_Order_Book AS
SELECT o.Order_ID,
       t.Name AS Trader_Name,
       ss.Symbol,
       o.Order_Type,
       o.Quantity,
       o.Limit_Price,
```

```
        o.Status,  
        o.Timestamp  
FROM orders o  
JOIN Trader t ON o.Trader_ID = t.Trader_ID  
JOIN Stock_Base sb ON o.Stock_ID = sb.Stock_ID  
JOIN Stock_Symbol ss ON sb.Symbol = ss.Symbol  
WHERE o.Status IN ('PENDING','OPEN');  
  
SELECT * FROM vw_Order_Book ORDER BY Timestamp;
```

29: Transaction — Execute a Trade Atomically

```
START TRANSACTION;  
  
INSERT INTO Trade VALUES (1006, 2870.00, 8, NOW());  
  
UPDATE orders  
SET Status = 'FILLED'  
WHERE Order_ID = 211 AND Trader_ID = 5;  
  
ROLLBACK;  
  
INSERT INTO orders (  
    order_id, trader_id, stock_id, quantity,  
    limit_price, status, timestamp, order_type  
)  
VALUES (213, 101, 1, 8, 2870.00, 'FILLED', NOW(), 'SELL');  
  
INSERT INTO orders (  
    order_id, trader_id, stock_id, quantity,  
    limit_price, status, timestamp, order_type  
)
```

```
VALUES (211, 102, 1, 8, 2800.00, 'FILLED', NOW(), 'BUY');  
  
INSERT INTO trade_order VALUES (1006, 211), (1006, 213);  
  
INSERT INTO System_Audit_Log (Actor_ID, Action_Type,  
Target_Entity_ID)  
VALUES (1, 'TRADE_EXECUTED', 1006);  
  
COMMIT;
```

34: Create Indexes on High-Frequency Query Columns

```
CREATE INDEX idx_Order_Status ON orders (Status);  
  
CREATE INDEX idx_Order_Timestamp ON orders  
(Timestamp);  
  
CREATE INDEX idx_Audit_Actor ON System_Audit_Log  
(Actor_ID);  
  
CREATE INDEX idx_Audit_Timestamp ON System_Audit_Log  
(Action_Timestamp);  
  
CREATE INDEX idx_Portfolio_Composite ON Portfolio  
(Trader_ID, Stock_ID);  
  
SELECT  
    indexname,  
    indexdef  
FROM pg_indexes  
WHERE tablename = 'orders';
```

35: CASE WHEN — Classify Traders by Balance Tier

```
SELECT Trader_ID, Name, Balance,
       CASE
         WHEN Balance >= 300000 THEN 'Platinum'
         WHEN Balance >= 150000 THEN 'Gold'
         WHEN Balance >= 50000 THEN 'Silver'
         ELSE 'Bronze'
       END AS Tier
FROM Trader
ORDER BY Balance DESC;
```

36: CTE (Common Table Expression) — Most Actively Traded Stock

```
WITH StockVolume AS (
  SELECT sb.Stock_ID, ss.Symbol, ss.Company_Name,
         SUM(t.Execution_Price * t.Quantity) AS Total_Value
  FROM Trade t
  JOIN Trade_Order to1 ON t.Trade_ID = to1.Trade_ID
  JOIN orders o ON to1.Order_ID = o.Order_ID
  JOIN Stock_Base sb ON o.Stock_ID = sb.Stock_ID
  JOIN Stock_Symbol ss ON sb.Symbol = ss.Symbol
  GROUP BY sb.Stock_ID, ss.Symbol, ss.Company_Name
)
SELECT Symbol, Company_Name, Total_Value
FROM StockVolume
WHERE Total_Value = (SELECT MAX(Total_Value) FROM
StockVolume);
```

37: GROUP BY WITH ROLLUP — Hierarchical Order Count Summary

```
SELECT
    COALESCE(t.name, '-- GRAND TOTAL --') AS trader_name,
    COALESCE(o.status, 'ALL') AS order_status,
    COUNT(o.order_id) AS order_count
FROM trader t
JOIN orders o ON t.trader_id = o.trader_id
GROUP BY ROLLUP(t.name, o.status)
ORDER BY t.name, o.status;
```

38: Comprehensive Dashboard Query — Full Trader Activity Report

```
SELECT t.Trader_ID,
       t.Name,
       t.Balance,
       COUNT(DISTINCT o.Order_ID) AS Total_Orders,
       SUM(CASE WHEN o.Status='FILLED' THEN 1 ELSE 0
END) AS Filled,
       SUM(CASE WHEN o.Status='PENDING' THEN 1 ELSE 0
END) AS Pending,
       SUM(CASE WHEN o.Status='CANCELLED' THEN 1 ELSE
0 END) AS Cancelled,
       COUNT(DISTINCT p.Stock_ID) AS Stocks_Held,
       COALESCE(SUM(p.Quantity), 0) AS Total_Shares
FROM Trader t
LEFT JOIN orders o ON t.Trader_ID = o.Trader_ID
LEFT JOIN Portfolio p ON t.Trader_ID = p.Trader_ID
```

```
GROUP BY t.Trader_ID, t.Name, t.Balance  
ORDER BY Filled DESC, t.Balance DESC;
```

39.Total trading volume per stock

```
SELECT o.Stock_ID, SUM(t.Quantity) AS Total_Volume  
FROM Trade t  
JOIN Trade_Order to2 ON t.Trade_ID = to2.Trade_ID  
JOIN orders o ON to2.Order_ID = o.Order_ID  
GROUP BY o.Stock_ID;
```

40.Most actively traded stocks

```
SELECT o.Stock_ID, COUNT(*) AS Trade_Count  
FROM Trade t  
JOIN Trade_Order to2 ON t.Trade_ID = to2.Trade_ID  
JOIN orders o ON to2.Order_ID = o.Order_ID  
GROUP BY o.Stock_ID  
ORDER BY Trade_Count DESC  
LIMIT 5;
```

41.Top 3 traders by trading volume

```
SELECT o.Trader_ID, SUM(t.Quantity) AS Volume  
FROM Trade t  
JOIN Trade_Order to2 ON t.Trade_ID = to2.Trade_ID  
JOIN orders o ON to2.Order_ID = o.Order_ID  
GROUP BY o.Trader_ID  
ORDER BY Volume DESC  
LIMIT 3;
```

42.Stocks never traded

```
SELECT s.Stock_ID  
FROM Stock_Base s  
LEFT JOIN orders o ON s.Stock_ID = o.Stock_ID  
WHERE o.Order_ID IS NULL;
```

43.Traders with no activity

```
SELECT t.Trader_ID, t.Name  
FROM Trader t  
LEFT JOIN orders o ON t.Trader_ID = o.Trader_ID  
WHERE o.Order_ID IS NULL;
```

44.Recent trading activity for a stock

```
SELECT *  
FROM trade  
WHERE trade_time >= NOW() - INTERVAL '1 day';
```

45.Average trading price of each stock

```
SELECT o.Stock_ID, AVG(t.Execution_Price) AS Avg_Price  
FROM Trade t  
JOIN Trade_Order to2 ON t.Trade_ID = to2.Trade_ID  
JOIN orders o ON to2.Order_ID = o.Order_ID  
GROUP BY o.Stock_ID;
```